# Création d’un capteur connecté

Tâches & notation

## Règles de notation

- Chaque tâche rapporte 1 point, une tâche incomplète ou imparfaite peut être récompensée d’un demi point
- Le compteur et la commande reset donnés en exemple dans le guide de démarrage ne comptent pas pour les objectifs 6 à 16 : chaque groupe doit publier la donnée de son appareil et recevoir sa propre commande.

## A. Configuration (1 point) - Livrables 1 & 2

| | |
|---|---|
| ● | config.h correctement rempli. Le capteur vise la passerelle du TP et porte un nom sensor_<lettre du groupe> conforme aux règles et unique dans la classe. |

## B. Mise en route suivie jusqu'au bout (4 points) - Livrable 1

| | |
|---|---|
| ● | Le capteur est en ligne dans l'IHM. Il s'enregistre auprès de la passerelle, et la tuile du groupe affiche son identifiant et son heartbeat. |
| ● | Le compteur est publié et affiché. La tuile affiche un compteur qui augmente toutes les 2s. |
| ● | La commande reset fonctionne de bout en bout. Le bouton remet le compteur à 0, et l'affichage ne change qu'à la réception de la confirmation du capteur. |
| ● | Le compteur est republié à l'ouverture de session. Après un redémarrage de l'IHM ou de la passerelle, la tuile retrouve la valeur courante du compteur. |

## C. Protocole côté capteur, C++ (7 points + 2 bonus) - Livrable 2

| | |
|---|---|
| 6 | Le capteur lit une vraie donnée. Le capteur lit une broche analogique ou numérique (câblée ou non) et publie la valeur lue. |
| BONUS (+2) | Le capteur lit un DHT11. La température et l'humidité du DHT11 sont publiées et affichées dans la tuile, dans des unités lisibles. |
| 7 | Les subjects du groupe sont déclarés proprement. Au moins un status_ publié et une cmd_ (ou set_) reçue, chaque préfixe correspondant au rôle du message. |
| 8 | Contrôle de l’état d’allumage de la LED RGB intégrée à la carte ESP32-C3 via un cmd_. |
| 9 | Contrôle de la couleur de la LED RGB intégrée à la carte ESP32-C3 via un set_. |
| 10 | Chaque payload est une structure packed vérifiée à la compilation. Chaque format a une taille fixe, garantie à la compilation.<br>Les champs portent leur unité dans leur nom. Chaque grandeur physique ou durée est un entier dont le nom donne l'unité (centiDegrees, periodMs…). loop() ne bloque jamais. Aucune attente dans loop() ni dans les handlers. |

| | |
|---|---|
| 11 | L'état est republié à l'ouverture de session. À chaque ouverture de session, le capteur republie tout son état. L'état et la couleur de la LED sont également republiés au démarrage. |
| 12 | Le(s) handler de commande (cmd_) est/sont robuste et confirment par un status_. Idem pour les set_ Un message de taille inattendue est ignoré, et chaque commande appliquée est confirmée par un status_. |

## D. Tuile Godot (4 points) - Livrable 2

| | |
|---|---|
| 13 | Les noms de subjects sont identiques des deux côtés. La tuile et le croquis utilisent exactement les mêmes subjects, construits à partir de DEVICE_NAME.<br>La tuile respecte le contrat du payload. Un payload de taille inattendue est ignoré, et la valeur affichée est correcte, dans une unité lisible. |
| 14 | La couleur de la LED est modifiable par l'IHM, que ça soit via un slider ou par génération d'une couleur aléatoire. On peut demander l'allumage et l'extinction de la LED depuis la tuile d'IHM. |
| 15 | L'affichage ne change que sur réception du status_. Une action de l'utilisateur ne modifie jamais directement l'affichage. |
| 16 | Les éléments ajoutés gèrent la déconnexion. Capteur déconnecté, ses valeurs apparaissent comme périmées (ex : grisées) et ses commandes sont indisponibles. Tout revient à la reconnexion. |

## E. Qualité de code et documentation (4 points) - Livrable 1 & 2

| | |
|---|---|
| 17 | Aucun nombre magique. Toute valeur numérique autre que 0 et 1 est une constante nommée, en C++ comme en GDScript. |
| 18 | Une convention de nommage par langage, appliquée sans exception. C++ : kSub... pour les subjects, PascalCase pour les types, camelCase pour le reste.<br>GDScript : UPPER_SNAKE_CASE pour les constantes, snake_case pour le reste, préfixe _ pour ce qui est privé, pour les callbacks du moteur (_ready, _process) et pour les paramètres inutilisés.<br>Tout autre choix devra être défendu mais n'est en aucun cas proscrit tant que la cohérence du code et sa lisibilité son assurés. |
| 19 | Chaque fonction ajoutée est documentée.<br>Chaque fonction ajoutée a un commentaire d'en-tête qui dit son rôle.<br>Il ne reste ni TODO, ni code commenté, ni trace de débogage. |
| 20 | Le README documente le contrat. Il décrit chaque subject (nom, sens, champs avec type, unité et taille), en accord avec le code. |

# Création d'un capteur connecté

Tâches & notation

## Livrables

### Livrable 1 :

Si vous souhaitez alléger votre tuile d'IHM après la fin de la mise en route, assurez-vous de maintenir une archive ou de tagger un repo Github de votre code comprenant les éléments de la prise en main.

Vous pouvez décider de travailler par dessus, sans repartir d'un boilerplate, auquel cas vous serez noté sur le contenu du livrable 2 qui contiendra également le contenu du premier.

Ce livrable contient :

- code source Arduino (C++), prêt à compiler (dossier contenant le fichier ".ino" et "config.h")
- code IHM, prêt à être lancé (dossier HMI contenant entre autres "project.godot")
- README.md

Vous pouvez le rendre sous forme :

- d'un repo Git accessible (Github public par exemple)
- d'une archive "groupe_<lettre>_livrable_1.zip" contenant les deux codes, chacun dans un sous-dossier respectif, et le README.md

### Livrable 2 :

Le code final de votre capteur et de votre tuile IHM.

Ce livrable contient :

- code source Arduino (C++), prêt à compiler (dossier contenant le fichier ".ino" et "config.h")
- code IHM, prêt à être lancé (dossier HMI contenant entre autres "project.godot")
- README.md

Vous pouvez le rendre sous forme :

- d'un repo Git accessible (Github public par exemple)
- d'une archive "groupe_<lettre>_livrable_2.zip" contenant les deux codes, chacun dans un sous-dossier respectif, et le README.md